

FlashAttention-4：用于非对称硬件扩展的算法与内核流水线协同设计

Ted Zadouri^{*1,6} Markus Hoehnerbach^{*2} Jay Shah^{*3}

Timmy Liu⁴ Vijay Thakkar^{2,5} Tri Dao^{1,6}

¹ 普林斯顿大学 ²Meta ³Colfax Research ⁴NVIDIA ⁵佐治亚理工学院 ⁶Together AI

Abstract

注意力机制作为无处不在的 Transformer 架构的核心层，是大型语言模型和长上下文应用的性能瓶颈。尽管（warp specialization）为 Hopper GPU 优化了注意力计算，但它主要针对 H100 架构。AI 行业已迅速转向部署基于 Blackwell 架构的系统（如 B200 和 GB200），这些系统由于硬件的非对称扩展而表现出根本不同的性能特性：张量核心（tensor core）吞吐量翻倍，而其他功能单元（共享内存带宽、指数运算单元）的扩展较慢或保持不变。我们开发了多项技术来解决 Blackwell GPU 上这些不断变化的瓶颈：(1) 重新设计的流水线，充分利用完全异步的 MMA（矩阵乘加）操作和更大的分块尺寸；(2) 软件模拟的指数运算和条件式 softmax 重缩放，减少非矩阵乘法操作；(3) 利用张量内存（tensor memory）和 2-CTA MMA 模式，减少后向传播中的共享内存流量和原子加操作。我们展示了 200 GPU 上使用 BF16 精度可达到 cuDNN 9.13 的 1.3 倍加速和 Triton 的 2.7 倍加速，最高达到 1613 TFLOPS/s（71% 利用率）。除算法创新外，我们完全使用嵌入 Python 的 CuTe-DSL 实现了 FlashAttention-4，与传统基于 C++ 模板的方法相比，编译时间缩短了 20–30 倍，同时保持了完整的表达能力。

1 引言

Transformer 架构 [27] 至今仍是几乎所有 AI 应用的主要基础架构，从大型语言模型 [2] 到视觉 [8] 和多模态系统。对于 Transformer 而言，注意力机制构成了主要的计算瓶颈——查询（query）和键（key）之间计算的自注意力得分在序列长度上呈二次方扩展。将注意力扩展到更长的上下文可以解锁新的能力，例如跨多文档的推理 [10, 24]、对整个代码库的建模 [22] 以及高分辨率视频处理 [3, 11]。与此同时，加速器硬件持续快速演进 [19]，每一代产品都提供了显著更高的峰值计算吞吐量。然而，这种演进是非对称的：矩阵乘法单元积极扩展，而其他功能单元（如内存带宽和专用计算单元）扩展较慢，造成了日益不平衡的硬件流水线，需要深入的算法协同设计。

这引发了人们对通过深度整合 GPU 硬件特性来加速注意力的算法创新的持续关注。Dao et al. [6] 提出了 FlashAttention，通过新颖的分块技术和算子融合消除了对慢速全局内存的中间读写操作。Dao [5] 将其重构为，提高了 GPU 占用率。Shah et al. [23] 进一步将算法适配到 Hopper GPU 成为 FlashAttention-3，通过线程束特化利用异步执行并加入了 FP8 支持。近期工作还探索了低精度注意力：SageAttention [13] 通过 INT8 量化实现加速，SageAttention2 [12] 使用 INT4/FP8 量化进一步扩展，SageAttention3 [14] 在 Blackwell 消费级 GPU 上实现了 FP4 量化。然而，这些方法主要针对消费级 GPU，而大多数 AI 计算部署在数据中心 GPU 上。同时，Hopper H100 架构，而 AI 行业已迅速转向部署基于 Blackwell 的数据中心系统 [19]（如 B200 和 GB200），这些 GPU 代表了具有根本不同性能特性的新一代架构。

加速器演进的一个关键趋势是硬件单元的非对称扩展。尽管 Blackwell B200 的张量核心吞吐量比 Hopper H100 翻了一番（FP16/BF16 达 2.25 PFLOPS vs 1 PFLOPS），其他功能单元（共享内存带宽、指数运算单元以及整数/浮点 ALU）的扩展较慢或保持不变。因此，非 MMA 资源成为瓶颈。我

*同等贡献

们的屋顶线分析 (Sections 3.1 and 3.2) 显示, 对于 Blackwell 上典型的注意力工作负载, 出人意料地, 共享内存流量和指数运算现在主导了执行时间, 超过 MMA 计算的 25–60%。此外, Blackwell 引入了新的架构特性: 每 SM 拥有 256 KB 的张量内存 (TMEM) 用于存储张量核心运算的中间结果; 128×128 的 MMA 分块 (是 Hopper 的 64×128 的两倍面积); 以及完全异步的张量核心操作, 直接将结果写入 TMEM。简单地将现有注意力算法移植到这个新硬件上, 要么会留下显著的性能余量, 要么由于 Hopper MMA 指令缺乏前向兼容性而根本无法实现。

为此, 我们提出了 FlashAttention-4, 协同设计算法和内核实现, 以解决现代 GPU 架构中不断变化的瓶颈。我们不把硬件视为均匀的计算资源, 而是通过算法创新显式地识别并缓解非矩阵乘法单元的瓶颈:

1. 重新设计的流水线以实现最大重叠: 我们为前向传播和后向传播开发了新的软件流水线, 充分利用 Blackwell 完全异步的 MMA 操作和更大的分块尺寸, 以最大化张量核心、softmax 计算和内存操作之间的重叠。
2. 指数运算单元瓶颈缓解: 对于前向传播, 我们使用 FMA 单元上的多项式近似实现软件模拟指数函数, 提高指数运算吞吐量。我们还引入了条件式 softmax 重缩放, 跳过不必要的重缩放操作。
3. 共享内存流量减少: 对于后向传播, 我们利用张量内存存储更多中间结果, 减少共享内存流量。我们还利用 Blackwell 的 2-CTA MMA 模式, 使每个 CTA 只需装载操作数 B 的一半, 进一步减少共享内存流量, 借此重构 dQ 步骤以将全局原子规约次数减半。我们还实现了性能开销极小的确定性执行模式, 为强化学习应用提供可复现的训练。
4. 改进的调度和资源分配: 我们针对 Blackwell 的资源约束和更大的分块尺寸开发了新的 CTA 调度策略和寄存器分配方案。

除算法创新外, 我们完全使用嵌入 Python 的 CuTe-DSL 实现了 FlashAttention-4, 与传统基于 C++ 模板的方法相比, 编译时间缩短了 20–30 倍, 同时保持了完整的表达能力。该框架显著提升了开发者的生产力, 降低了入门门槛, 使研究人员无需深厚的 C++ 模板元编程专业知识即可快速原型化和部署新的注意力变体。

为实证验证我们的方法, 我们在 B200 GPU 上对, 结果表明: (1) BF16 精度下比 cuDNN 最高实现 1.3 倍加速, 比 Triton 实现 2.7 倍加速; (2) 我们在性能瓶颈已转移的资源上达到接近峰值的利用率, 最高达到 ~1600 TFLOPS (理论最大值的 71%); (3) 对于长序列,。

我们以宽松许可证开源了 FlashAttention-4, 并正在努力将其集成到主流库中, 以惠及更多研究人员和开发者。代码可访问: https://github.com/Dao-AI-Lab/flash-attention/tree/main/flash_attn/cute

2 背景

2.1 多头注意力

设 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ 为与单个注意力头相关的查询、键和值输入序列, 其中 N 为序列长度, d 为头维度。注意力输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$ 的计算如下:

$$\begin{aligned}\mathbf{S} &= \alpha \mathbf{QK}^T \in \mathbb{R}^{N \times N}, \\ \mathbf{P} &= \text{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \\ \mathbf{O} &= \mathbf{PV} \in \mathbb{R}^{N \times d},\end{aligned}$$

其中 softmax 按行应用, $\alpha = 1/\sqrt{d}$ 为缩放因子。实践中, 为了数值稳定性, 我们从 $\mathbf{S}$ 中减去 $\text{rowmax}(\mathbf{S})$ 。对于多头注意力 (MHA), 每个头有自己的投影集合, 该计算在多个头和批次上并行化。

给定输出梯度 $\mathbf{dO} \in \mathbb{R}^{N \times d}$ ，后向传播计算：

$$\begin{aligned}\mathbf{dV} &= \mathbf{P}^\top \mathbf{dO}, \quad \mathbf{dP} = \mathbf{dOV}^\top, \\ \mathbf{dS} &= \text{dsoftmax}(\mathbf{dP}), \\ \mathbf{dQ} &= \alpha \mathbf{dSK}, \quad \mathbf{dK} = \alpha \mathbf{dS}^\top \mathbf{Q},\end{aligned}$$

其中 $\text{dsoftmax}(\mathbf{dP})$ 表示逐行的 softmax 梯度 $\mathbf{ds} = (\text{diag}(p) - pp^\top) \mathbf{dp}$ ，其中 $p = \text{softmax}(s)$ 。

2.2 GPU 硬件特性与执行模型

我们描述 GPU 执行模型中与，重点关注 NVIDIA Blackwell 架构 (B200 & GB200)。我们着重介绍与之前 Hopper 架构的主要差异，这些差异推动了。

内存层次结构： GPU 的内存组织为一个数据位置的层次结构，容量与带宽成反比。全局内存 (GMEM)，也称为 HBM，是所有流多处理器 (SM) 均可访问的片外 DRAM。来自 GMEM 的数据被透明地缓存在片上 L2 缓存中。其次，每个 SM 包含一个小型的、程序员可管理的、高度分组的缓存，称为共享内存 (SMEM)。最后是每个 SM 内的寄存器文件。

Blackwell 引入了一种新的内存层级，称为张量内存 (TMEM)，这是每 SM 256 KB 的片上内存，专门用于存储张量核心操作的中间结果。与共享内存不同，TMEM 是线程束同步的，与张量核心紧密耦合，使矩阵乘加 (MMA) 单元能够直接将输出写入 TMEM 而无需消耗寄存器。这缓解了困扰 Hopper 内核的极端寄存器压力，并支持更大的分块尺寸。TMEM 以 32 列 (16 KB) 的粒度分配，需要程序员显式管理分配、释放和数据移动。

线程层次结构： GPU 的编程模型围绕称为线程的执行单元的逻辑分组组织。从最细粒度到最粗粒度，线程层次结构由线程、线程束 (32 个线程)、线程束组 (4 个连续线程束)、线程块 (即协作线程数组或 CTA)、线程块集群和网格构成。同一 CTA 中的线程被协同调度在同一 SM 上，同一集群中的 CTA 被协同调度在同一 GPC 上。SMEM 可由 CTA 内的所有线程直接寻址，而每个线程最多拥有 256 个寄存器 (RMEM)，这些寄存器对每个线程是私有的。

张量核心与增强的异步性： Blackwell 配备了第五代张量核心，处理的分块比之前架构更大。每条 MMA 张量核心指令处理 $128 \times N$ 的分块 (通常 $N = 128$ 或 256)，而 Hopper 上为 $64 \times N$ 。关键的是，Blackwell MMA 异步地将输出直接写入 TMEM，而 Hopper MMA 写入寄存器。这种完全的异步性使计算与其他操作之间有更好的重叠，因为 MMA 单元不再因寄存器回写而阻塞。

硬件对异步性的支持使得线程束特化内核成为可能，CTA 的线程束被划分为生产者或消费者角色，分别只发出数据移动或计算指令 [1]。

2-CTA 张量核心： Blackwell 支持 2-CTA 张量核心 MMA 模式，其中同一线程块集群内的 CTA 对协同执行单个 MMA，使该操作能够读写来自两个 CTA 的张量内存。CTA 对中的一个线程发起 MMA，但对端 CTA 在操作进行时必须已启动并保持活跃。与单 CTA MMA 将 M 维度限制在 128 相比，配对模式通过在 M 维度上跨 CTA 对划分 A 分块和累加器，并在 N 维度上跨两个 CTA 划分 B 分块，支持 $M = 128$ 或 256 ，使得每个 CTA 只需在自己的共享内存中装载 B 的一半，而硬件在乘法过程中处理组合后的 B 分块。这减少了冗余的共享内存容量和带宽，但由于这些操作跨 CTA 对访问张量内存，内核必须以固定对的方式启动 CTA，并在整个内核中为张量内存和张量核心操作使用一致的 2-CTA 模式。

性能瓶颈的转移： Blackwell 反映的一个关键趋势是张量核心吞吐量比其他功能单元增长更快。与 Hopper 相比，Blackwell 将 FP16/BF16 张量核心吞吐量翻了一番 (2.25 PFLOPS [19] vs 1 PFLOPS [17] 每 GPU)，但共享内存带宽和指数运算单元吞吐量保持不变或扩展较慢。这种不平衡将性能瓶颈从矩阵乘法转移到了共享内存流量和非矩阵乘法操作 (如 softmax)。正如我们在 Sections 3.1 and 3.2 中的屋顶线分析所示，这需要精心的内核设计来最大化 MMA 操作与这些瓶颈资源之间的重叠。

B200 (和 GB200) 上几个硬件组件的吞吐量如下所示。

1. 张量核心：BF16 MMA 的吞吐量为每时钟每 SM 8192 次操作，是 Hopper 的 4096 次的两倍。这可以从理论最大 FLOPS 推导：2.25 PFLOPS / 1850 MHz 时钟速度 / 148 SMs = 每时钟每 SM 8192 次操作。
2. 指数运算单元。B200 和 GB200 上的多功能单元 (MUFU) 每时钟每 SM 可执行 16 次操作，与 Hopper 相同 [18]。值得注意的是，B300 和 GB300 GPU 将指数吞吐量翻倍至每时钟每 SM 32 次操作，不过这些 GPU 在撰写本文时尚未广泛上市。
3. SMEM：读取吞吐量为每时钟每 SM 128 字节，与 Hopper 相同，通过微基准测试测得 [15]。我们看到，Blackwell 上的 MMA 吞吐量是 Hopper 的两倍，但其他硬件单元不一定以相同速率变快。这反映了加速器设计的一个更广泛趋势：在相似功耗/硅片面积约束下，通过提高最重要组件（通常是矩阵乘法单元）的吞吐量来获得更高性能。

3 算法

3.1 注意力前向传播

我们首先进行屋顶线 (Roofline) 分析，展示注意力前向传播的瓶颈，这为我们新的流水线设计提供了动力，同时也推动了。

3.1.1 资源分析

我们通过首先分析屋顶线来为内核设计和优化提供直觉，分析基于矩阵乘法单元（张量核心）、共享内存 (smem) 和指数运算单元的吞吐量。需要注意的是，这是一个简化分析，未考虑 GPU 中的所有资源（例如浮点数学、寄存器带宽、L2 带宽），但足以识别性能瓶颈。

设 $\mathbf{Q}$ 和 $\mathbf{K}$ 的序列长度维度上的块形状为 $M \times N$ ，头维度为 d 。我们分析计算和内存流量需求以识别性能瓶颈。

MMA 计算。前向传播每次迭代执行两次矩阵乘加 (MMA) 操作： $\mathbf{QK}^\top$ （从 $M \times d$ 和 $d \times N$ 输入计算 $M \times N$ 输出）和 $\mathbf{PV}$ （从 $M \times N$ 和 $N \times d$ 输入计算 $M \times d$ 输出）。每次 MMA 需要 $2MNd$ 次浮点操作。张量核心吞吐量为每周期 8192 次 FLOP，总计算时间为

$$T_{\text{MMA}} = \frac{4MNd}{8192} \text{ 周期。} \quad (1)$$

共享内存流量。两次 MMA 中，一次是共享-共享 (SS) 类型，两个操作数均从共享内存读取 ($\mathbf{QK}^\top$)，另一次是张量-共享 (TS) 类型，操作数 A 从张量内存读取，操作数 B 从共享内存读取 ($\mathbf{PV}$)。由于每条 MMA 指令处理 128×128 的分块，计算 $M \times N$ 输出需要 $\lceil M/128 \rceil \times \lceil N/128 \rceil$ 条 MMA 指令。关键的是，当需要多条 MMA 指令时，共享内存操作数会被多次读取。

对于 $\mathbf{QK}^\top$ (SS)，计算 $M \times N$ 输出需要 $\lceil M/128 \rceil \times \lceil N/128 \rceil$ 条 MMA 指令，每条从共享内存读取 $128 \times d$ 的 $\mathbf{Q}$ 块和 $d \times 128$ 的 $\mathbf{K}^\top$ 块。总共享内存读取量为 $\lceil M/128 \rceil \times \lceil N/128 \rceil \times (128d + 128d) = \lceil M/128 \rceil \lceil N/128 \rceil \times 256d$ 个元素。对于 $\mathbf{PV}$ (TS)，计算 $M \times d$ 输出需要 $\lceil M/128 \rceil \times \lceil d/128 \rceil$ 条 MMA 指令，每条从共享内存读取 $N \times 128$ 的 $\mathbf{V}$ 块，总计 $\lceil M/128 \rceil \times \lceil d/128 \rceil \times 128N$ 个元素。以每元素 2 字节 (bf16)，每周期 128 字节带宽计，共享内存 (T_{smem}) 读取时间为

$$= 2 \left\lceil \frac{M}{128} \right\rceil \left\lceil \frac{N}{128} \right\rceil 256d + 2 \left\lceil \frac{M}{128} \right\rceil \left\lceil \frac{d}{128} \right\rceil 128N = \frac{3MNd}{8192} \text{ 周期} \quad (2)$$

（假设 M 、 N 、 d 均为 128 的倍数）。

指数运算单元。指数运算单元执行 softmax 计算所需的逐元素操作。前向传播需要对 $M \times N$ 个值（对应注意力矩阵 $\mathbf{S}$ ）进行指数运算。吞吐量为每周期 16 次操作，指数运算单元所需时间为

$$T_{\text{exp}} = \frac{MN}{16} \text{ 周期。} \quad (3)$$

表 1 总结了两种典型分块配置的分析结果。对于 $M = N = d = 128$ ，资源相对均衡，共享内存 (768 周期) 略低于 MMA 计算和指数运算单元（均为 1024 周期）。对于更大的分块尺寸 $M = 256, N = d = 128$ ，由于 MMA 操作数被多次读取，共享内存流量增加至 1536 周期，而 MMA 计算和指数运算单元翻倍至 2048 周期。该分析为我们的内核设计提供了动力：(1) 使用大分块并最

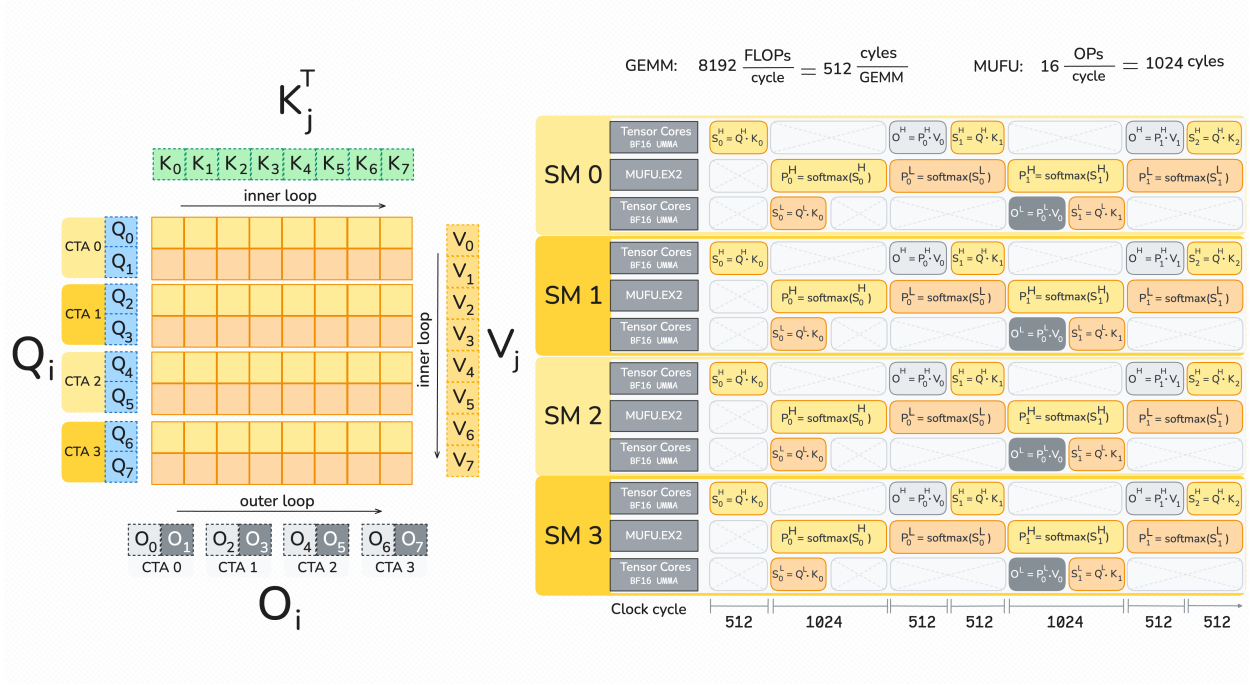


Figure 1: FlashAttention-4 前向传播流水线。上标 H 表示对应“高”Q 块的矩阵，上标 L 表示对应“低”Q 块的矩阵。每个 Q 块对应 128 个查询 token。

资源	128 ³	256 × 128 ²
MMA 计算	1024	2048
共享内存	768	1536
指数运算单元	1024	2048

Table 1: 注意力前向传播的屋顶线分析（单位：周期）。对于两种分块尺寸，MMA 计算和指数运算单元均为主要瓶颈。

大化 MMA 操作与 softmax 计算之间的重叠；(2) 利用其他硬件单元提高指数运算吞吐量；(3) 减少不必要的非矩阵乘法操作时间。

3.1.2 重叠矩阵乘法与 softmax 的新流水线

由于 Blackwell 架构再次将张量核心 FLOPS 翻倍，注意 softmax 与张量核心操作的重叠比在 Hopper 上更为关键。我们遵循类似 FA-3 的乒乓调度方式，每个线程块计算两块输出。当一块的张量核心操作执行时，另一块计算 softmax。Hopper 张量核心以交错模式将累加器保存在寄存器中（每行四个线程），而 Blackwell 张量核心将累加器保存在张量内存中。此外，Blackwell 上单个累加器块为 128×128 个元素，而 Hopper 的块大小为 64×128 。

在这些块上分配工作的自然方式是使用两组 128 个线程的线程束组，每个线程处理整行。这消除了用于规约行最大值的线程束间洗牌操作，以及每个线程的多个统计寄存器的需求。与 FA-3 一样，我们显式同步两个 softmax 线程束组，使其在关键区（即指数计算部分）不重叠。每个 softmax 线程束组首先将整行加载到寄存器，然后计算最大值，再计算 softmax（即减去最大值、重缩放、指数化、转换为输入精度），最后计算行和。

与 FA-3 的另一个区别是，由于我们通过张量内存而非寄存器文件传输 $\mathbf{P}$ ，我们可以将输出的重缩放解耦到独立的“校正”线程束组，从而将其从关键路径中移除。

为实现这种流水线重叠，可以选择多种张量内存分区方案。所有方案都必须为两块输出分配空

间，在头维度为 128 时，留下一半张量内存存储 **S** 和 **P**。这些内存可以存储两个 **S** 副本或四个 **P** 副本（假设输入为 FP16 或 BF16 张量核心）。这给我们留下了大致两种分区选项：一块 **S** 和两块 **P**，或两块 **S** 与 **P** 重叠。我们选择后者，因为它允许我们立即从计算两块 **S** 开始软件流水线。这也留下了一些张量内存用于向校正线程束组传递重缩放统计信息。

较大的 Blackwell 分块尺寸和所选的线程分配方案引入了一个问题：除非从张量内存重新加载，否则我们必须在寄存器中保存整行的 128 个元素。由于我们使用两个 softmax 线程束组、一个校正线程束组和一个驱动张量核心及 TMA 单元的线程束组，为 softmax 分配足够的寄存器并防止寄存器溢出至关重要。对于 BF16 输入数据类型，我们需要 128 个寄存器存储输入，可能还需要 64 个寄存器存储输出（加上杂项和临时寄存器）。为降低寄存器压力，我们分阶段存储 **P**：前四分之三存储一次（并触发相应的 MMA 操作），最后四分之一单独存储。

3.1.3 指数函数的软件模拟

指数运算吞吐量瓶颈。在现代 GPU 上，指数函数由多功能单元（MUFU）计算，其吞吐量远低于用于矩阵乘法的张量核心。在 B200 和 GB200 GPU 上，MUFU 每时钟每 SM 提供 16 次操作，而矩阵乘法为每时钟每 SM 8192 次操作。由于 softmax 计算需要大量指数求值，这种差异使指数函数成为注意力内核中的关键瓶颈。

通过多项式近似进行软件模拟。为提高指数运算吞吐量，我们使用浮点 FMA 单元实现了 2^x 的软件模拟，FMA 单元可与 MUFU 并行运行。我们使用经典的范围归约技术（Cody-Waite 方法）和多项式近似 [16]。关键思路是分解指数计算：

$$2^x = 2^{\lfloor x \rfloor} 2^{x - \lfloor x \rfloor} \quad (4)$$

其中 $\lfloor x \rfloor$ 是整数部分， $x - \lfloor x \rfloor \in [0, 1)$ 是小数部分。

整数部分 $2^{\lfloor x \rfloor}$ 可以利用 IEEE 754 浮点表示的位操作高效计算。由于指数字段直接表示 2 的幂次，计算 $2^{\lfloor x \rfloor}$ 等价于对指数位进行移位和加法操作，这可以使用整数 ALU 指令完成。

对于小数部分，我们用多项式近似 $2^{x_{\text{frac}}}$ ，其中 $x_{\text{frac}} \in [0, 1)$ ：

$$2^{x_{\text{frac}}} \approx \sum_{i=0}^n p_i x_{\text{frac}}^i \quad (5)$$

其中 $p_0 = 1.0$ ，其余系数选取为在 $[0, 1)$ 上最小化相对近似误差，使用 Sollya 软件包 [4] 计算。多项式求值使用 Horner 方法结合 FMA 指令，实现高吞吐量。

完整算法流程如下：

1. 将 x 截断至至少 -127 以避免下溢
2. 使用向下舍入模式计算 $\lfloor x \rfloor$ ：将 $2^{23} + 2^{22}$ 加到 x 上（将小数位强制到尾数中），然后以向下舍入模式减去
3. 计算小数部分： $x_{\text{frac}} = x - \lfloor x \rfloor$
4. 求多项式得 $2^{x_{\text{frac}}}$
5. 合并整数和小数部分：将 $\lfloor x \rfloor$ 移入指数字段并加上 $2^{x_{\text{frac}}}$ 的尾数位

通过将指数计算分布在 MUFU 和 FMA 单元上，这种方法有效提高了指数运算吞吐量，缓解了注意力计算中的关键瓶颈。

部分模拟。尽管多项式模拟提高了指数运算吞吐量，但它有一定代价：更多寄存器（用于保存中间值和系数）、更高的寄存器带宽消耗以及比 MUFU 指令更长的延迟。对所有指数求值使用模拟会增加寄存器压力并可能导致溢出，从而抵消吞吐量提升。相反，我们只对每个 softmax 行中的一部分元素（10–25%）应用模拟，其余通过硬件 MUFU.EX2 计算。确切比例根据给定分块配置下 MMA 和指数运算吞吐量的比率经验调优。

数值精度。表 2 比较了不同次数的多项式近似与硬件 MUFU.EX2 指令的精度，在 $[0, 1)$ 中的 400 万个随机输入上进行测量。我们报告两个指标：FP32 级别的误差（量化前）和 BF16 级别的误差（将 FP32 输出舍入到 BF16 后），均相对于 FP64 参考值测量。

方法	FP32 vs FP64		BF16 vs FP64	
	最大相对误差	平均相对误差	最大相对误差	平均相对误差
理想 (FP64→BF16)	—	—	3.89×10^{-3}	1.41×10^{-3}
硬件 MUFU.EX2	1.41×10^{-7}	3.04×10^{-8}	3.89×10^{-3}	1.41×10^{-3}
3 次多项式	8.77×10^{-5}	5.43×10^{-5}	3.90×10^{-3}	1.41×10^{-3}
4 次多项式	3.05×10^{-6}	1.84×10^{-6}	3.89×10^{-3}	1.41×10^{-3}
5 次多项式	1.44×10^{-7}	5.48×10^{-8}	3.89×10^{-3}	1.41×10^{-3}

Table 2: 在 $[0, 1)$ 上 2^x 多项式模拟的精度，在 400 万个随机输入上相对 FP64 参考值测量。FP32 列测量原始多项式输出；BF16 列测量舍入到 BF16 后的结果。对于所有 ≥ 3 次多项式，BF16 量化误差占主导地位。

在 FP32 级别，3 次多项式的最大相对误差为 8.8×10^{-5} ，大约是硬件的 600 倍。然而，舍入到 BF16 后，误差变得几乎无法区分：BF16 的量化误差 ($\sim 3.9 \times 10^{-3}$) 对所有 ≥ 3 次多项式的近似误差均占主导地位。3 次多项式在 99% 的输入上与硬件结果相差 1 个 BF16 ULP 以内，这对于注意力计算已经足够，因为 softmax 输出以 BF16 精度被消耗。更高次多项式可以缩小 FP32 差距：5 次多项式在最大相对误差上与硬件相差不到 2 倍，代价是每次求值额外增加两条 FMA 指令。

3.1.4 跳在线 softmax 重缩放

FlashAttention 在线 softmax。 FlashAttention 分块计算注意力 $\text{softmax}(QK^\top)V$ 以最小化内存流量。为了数值稳定性，算法在处理各块时维护运行统计信息。计算第 j 块时，令 $S_j = QK_j^\top$ 为该块的注意力分数。在线 softmax 算法跟踪：

$$m_j = \max(m_{j-1}, \text{rowmax}(S_j))$$

$$\ell_j = e^{m_{j-1}-m_j} \ell_{j-1} + \text{rowsum}(e^{S_j-m_j})$$

其中 m_j 是运行最大值， ℓ_j 是指数的运行和(归一化因子)。中间输出 O_j 更新为： $O_j = e^{m_{j-1}-m_j} O_{j-1} + e^{S_j-m_j} V_j$ 。重缩放因子 $e^{m_{j-1}-m_j}$ 通过在遇到更大值时重新归一化之前的结果来确保数值稳定性。

条件式重缩放。 步骤 $e^{m_{j-1}-m_j} O_{j-1}$ 需要一次向量乘法。我们做两个简单的观察：

1. 只有当 $m_j > m_{j-1}$ 时（即发现新的更大值时）才需要重缩放。
2. 我们可以在重缩放上容忍一些“余量”：只有当 $m_j - m_{j-1} > \tau$ 时才重缩放，其中 τ 是一个阈值（通常设为 $\log_2(256) = 8.0$ ，对应于 256.0 的重缩放因子）。只要我们跟踪统计信息（已执行的总缩放量），在最终仍可得到正确的分母以获得正确的最终输出。

在，我们将算法修改为：

$$O_j = \begin{cases} e^{m_{j-1}-m_j} O_{j-1} + e^{S_j-m_j} V_j & \text{若 } m_j - m_{j-1} > \tau \\ O_{j-1} + e^{S_j-m_{j-1}} V_j & \text{否则} \end{cases} \quad (6)$$

当 $m_j - m_{j-1} \leq \tau$ 时，我们跳过更新 m 并继续使用 m_{j-1} 。这保持了正确性，因为在计算结束时，所有累积值均通过真实最大值 m_{final} 和最终归一化因子 ℓ_{final} 重新归一化：

$$\text{输出} = \frac{1}{\ell_{\text{final}}} O_{\text{final}}$$

这一修改在保持数值精度的同时显著减少了重缩放操作的次数，因为最终归一化步骤会纠正跳过中间重缩放引入的小偏差。

实践中，为避免线程束分歧，当线程束中任何线程需要重缩放时，我们对整个线程束执行重缩放。

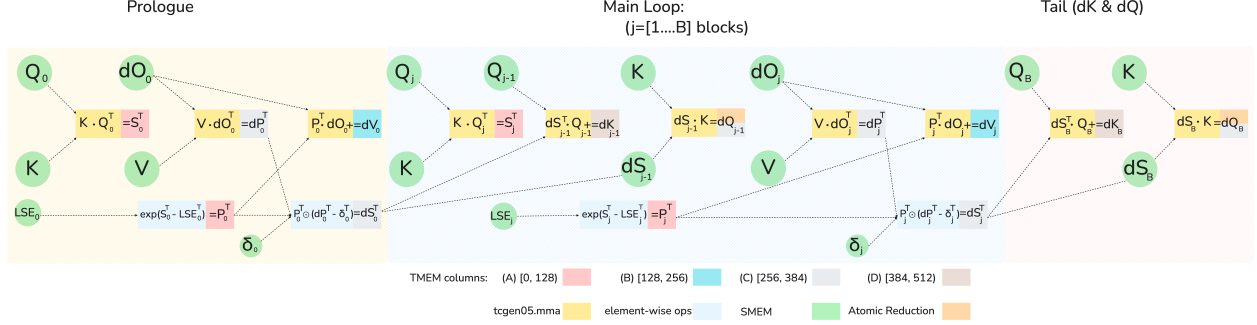


Figure 2: FlashAttention-4 后向传播计算图（5 次 MMA 操作 + 2 次逐元素操作），展示 1-CTA MMA 模式软件流水线在序言、主循环和尾声中的顺序。

3.2 注意力后向传播

3.2.1 资源分析

与前向传播类似，我们首先通过分析屋顶线为内核设计和优化提供直觉，分析基于矩阵乘法单元（张量核心）、共享内存（smem）和指数运算单元的吞吐量。

设 $\mathbf{Q}$ 和 $\mathbf{K}$ 的序列长度维度上的块形状为 $M \times N$ ，头维度为 d 。我们分析计算和内存流量需求以识别性能瓶颈。与前向传播不同，为简化 smem 周期数的公式，我们假设 $M = N = d = 128$ ，但保留变量名以明确表达。

MMA 计算。后向传播每次迭代执行五次矩阵乘加（MMA）操作。每次 MMA 涉及一个 $M \times N$ 矩阵、一个 $M \times d$ 矩阵和一个 $d \times N$ 矩阵（变化输出矩阵），需要 $2MNd$ 次浮点操作。张量核心吞吐量为每周期 8192 次 FLOP，总计算时间为

$$T_{\text{MMA}} = \frac{10MNd}{8192} \text{ 周期。} \quad (7)$$

共享内存流量。五次 MMA 中，三次—— $\mathbf{S}^\top = \mathbf{K}\mathbf{Q}^\top$ 、 $\mathbf{dP}^\top = \mathbf{V}\mathbf{dO}^\top$ 和 $\mathbf{dQ} = \mathbf{dS}\mathbf{K}$ ——是共享-共享（SS）操作，两个操作数均从共享内存读取；另外两次—— $\mathbf{dV} = \mathbf{P}^\top \mathbf{dO}$ 和 $\mathbf{dK} = \mathbf{dS}^\top \mathbf{Q}$ ——是张量-共享（TS）操作，操作数 A 从张量内存读取，操作数 B 从共享内存读取。SS MMA 从共享内存合计读取 $2Md + 3Nd + MN$ 个元素，TS MMA 从共享内存合计读取 $2Md$ 个元素。以每周期 128 字节的共享内存带宽和每元素 2 字节（bfl6）计，贡献为

$$T_{\text{smem,MMA}} = \frac{4Md + 3Nd + MN}{64} \text{ 周期。} \quad (8)$$

此外，算法将大小为 $M \times N$ 的中间梯度 $\mathbf{dS}$ 以 bfl6 写入共享内存，需要 $2MN$ 字节或 $MN/64$ 周期。大小为 $M \times d$ 的梯度 $\mathbf{dQ}$ 以 fp32（每元素 4 字节）写入共享内存，然后通过 TMA 读回用于规约，共 $8Md$ 字节的共享内存流量或 $Md/16$ 周期。

总共享内存访问时间（ T_{smem} ）因此为

$$\frac{4Md + 3Nd + MN}{64} + \frac{MN}{64} + \frac{Md}{16} \text{ 周期。} \quad (9)$$

指数运算单元。指数运算单元执行 softmax 及其梯度所需的逐元素操作（指数、对数及相关非线性函数）。后向传播需要对 $M \times N$ 个值（对应注意力矩阵 $\mathbf{S}$ 及相关项）进行指数运算。吞吐量为每周期 16 次操作，指数运算单元所需时间为

$$T_{\text{exp}} = \frac{MN}{16} \text{ 周期。} \quad (10)$$

表 3 总结了典型分块配置 $M = N = d = 128$ 的分析结果。共享内存流量时间 3328 周期超过了 MMA 计算时间（2560 周期）和指数运算单元时间（1024 周期），表明共享内存带宽是主要瓶颈，尽管程度不如全局内存流量那么严重。这为我们的内核设计提供了动力：最大化 MMA 操作与其他计算之间的重叠以隐藏共享内存延迟。

资源	周期数 ($N = d = 128$) 1-CTA ($M = 128$)	周期数 ($N = d = 128$) 2-CTA ($M = 256$)
MMA 计算	2560	2560
共享内存 (MMA 操作数)	2048	1536
共享内存 (dS 写入)	256	256
共享内存 (dS DSMEM)	0	384
共享内存 (dQ 写入 + 读出)	1024	512
共享内存总计	3328	2688
指数运算单元	1024	1024

Table 3: $M = N = d = 128$ 时注意力后向传播的屋顶线分析。共享内存流量是瓶颈，比 MMA 计算时间多约 30%。在 $M = 256, N = d = 128$ 的 2-CTA 设置中 (dQ MMA 例外，其 $M = N = 128, d = 256$)，共享内存流量比 MMA 计算时间多约 5%。

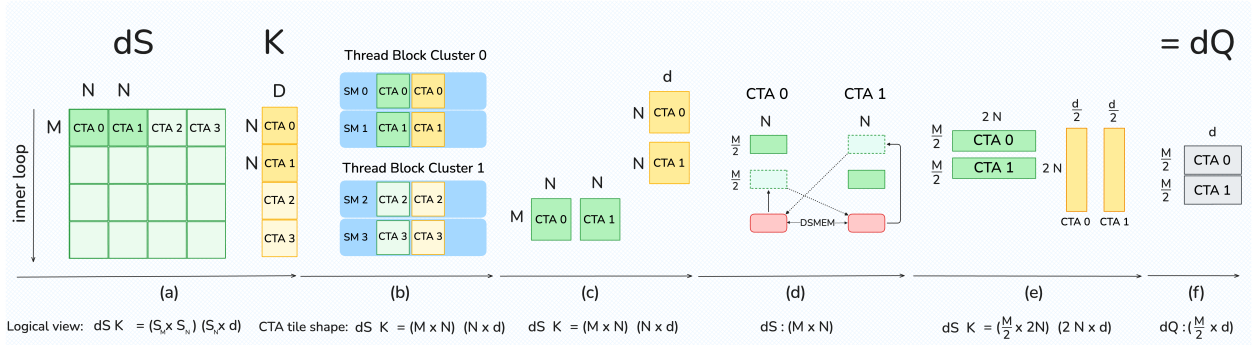


Figure 3: 在 2-CTA 后向传播 dQ 步骤中，CTA 对使用 DSMEM 交换 dS 块的一半，使每个 CTA 构成 $(\frac{M}{2} \times 2N)$ 操作数，并可使用翻倍规约运行 CTA 对 UMMA。

3.2.2 重叠矩阵乘法与 softmax 的新流水线

FlashAttention 后向传播执行五次 MMA 操作，对应于重新计算 S ，以及 QK （产生 dQ 和 dK ）和 PV （产生 dP 和 dV ）诱导的两组梯度计算。在 FA-3 中，累加器存储在寄存器中，寄存器是有限资源。这施加了显著的排序约束，实际上串行化了计算图，即按顺序计算 S, dP, dV, dQ, dK ，只有 TMA 加载可以大幅乱序执行。此外，算法类似：它沿 KV 序列长度维度迭代并以相对前向传播转置的方式计算值，因为 dV 和 dK 梯度计算需要从张量内存读取其一个操作数所要求的布局。dQ 通过原子操作累积。

在 FA-4 中，TMEM 与 FA-3 相比实现了额外的调度方案，提供了 MMA 与非 MMA 操作之间的显著重叠。具体地，与前向传播一样，我们试图隐藏 softmax 计算的延迟。在 FA-3 中，softmax 计算与 dP 的 MMA 重叠。从前一节可知，在 Blackwell 上我们需要同时运行至少两次 MMA 操作。

我们通过利用前一次迭代的 dQ 和 dK MMA 来实现这一点。这需要在加载、MMA、计算和规约操作之间仔细管理共享内存和张量内存资源。特别地，注意我们没有足够的张量内存来容纳五个累加器块。最多可以容纳四个 128×128 元素的块，且 dV 和 dK 需要累积，因此不能共享它们的空间。在我们的实现中， S 和 P 共享一个 tmem 块（偏移量为 0）， dP 、 dS 和 dQ 共享另一个。FA-4 后向传播的计算图见图 2。

3.2.3 2-CTA 后向传播：减少共享内存流量和全局原子加操作

即使采用改进的流水线且十个 GEMM 操作数中有两个常驻于张量内存，共享内存带宽仍然主导后向传播。在五个 GEMM 中，其余八个 BF16 操作数从共享内存加载以供张量核心使用，这种共享内存流量比张量核心计算多消耗约 30% 的周期。为进一步缓解这一瓶颈，我们使用 Blackwell

引入的 2-CTA MMA 模式，其中输出累加器在 M 维度上被划分。MMA 分块形状为 $M = 256$, $N = K = 128$ ，两个 CTA 充当单个更大的块：每个 CTA 装载并暂存操作数 B 的一半，只保留自己的累加器切片。

共享内存流量。后向传播有五个 GEMM，我们使用 MMA 分块形状 $M = 256$, $N = K = 128$ ，这大约将操作数 B 的共享内存流量减半。在 FlashAttention 后向传播中，每个 CTA 持有固定的 KV 块（在外层循环中跨 N 个 CTA 并行化），并在内层循环中流式处理 M 块。 dQ 的累积是外层循环中 KV 序列方向的规约，但 2-CTA MMA 只分割输出块，不分割规约轴，而 dQ MMA 的规约维度是 N ，它自然地跨 CTA 对分割。因此，每个 CTA 仍需要对其拥有的行进行完整规约。为解决规约轴上的这一冲突，我们使用分布式共享内存 (DSMEM) 在两个 CTA 之间交换 dS 的一半（它们在同一集群中）。这一方法对 dS 进行重新排列，使其沿非规约轴分区，每个 CTA 拥有其 $\frac{M}{2}$ 行并持有完整的 $2N$ 规约。因此，每个 CTA 的 dQ MMA 分块形状为 $(\frac{M}{2}, 2N)(2N, d)$ ，在张量内存中累积 $(\frac{M}{2}, d)$ 块。在 2-CTA MMA 模式中， S 、 dP 、 dV 和 dK 的 MMA 使用 $M = 256$ 的分块运行，而 dQ 使用 $M = 128$ 但翻倍的规约 $2N = 256$ 。我们随后相对于 1-CTA 变体重新排列了软件流水线以隐藏 DSMEM 延迟。我们在计算前一块的 dQ 之前先计算当前块的 dP 。 dQ 块足够小，可以与 P 一起放入 TMEM，重用与 S 相同的 TMEM 区域，因此我们不再像 1-CTA 模式那样为 dP 和 dQ 重用同一个 TMEM 区域。通过这种新的流水线排序，我们可以在计算前一次迭代块的 dQ MMA 时，并行计算当前块的逐元素 dS 。图 3 说明了 dQ 步骤的分解方式。

dQ 原子加操作。这种 dQ 分解的一个互补好处是，全局原子规约次数减半。原子更新引入不确定性且代价高昂，因为它们在内存循环的每次迭代中都会发生。因此，每个 CTA 只写入 dQ 块的一半，与 1-CTA 版本相比执行的全局原子规约次数减半。

3.2.4 确定性后向传播

我们的后向内核由于全局内存的跨 CTA 规约（对 dQ 一般情况，GQA 情况下还涉及 dK/dV ）而引入了梯度计算的不确定性。为确保可复现性并简化训练期间的可靠调试，我们还提供了确定性执行模式。我们采用的标准解决方案是使用信号量锁串行化全局规约。具体地，写入公共 dQ 块的每个 CTA 必须按预定义的顺序获取锁，执行规约，然后通过递增信号量计数器释放锁。

这种基于锁的方法影响性能的原因有两个：(1) 发出内存屏障以确保信号量写操作在设备范围内可见（正确的获取-释放语义所需）；(2) 每个 CTA 等待在公共 dQ 块上规约的前序 CTA 完成时引入停顿。在负载不均衡的情况下，CTA 顺序的朴素选择可能严重降低性能。总体上，我们在头和批次维度上进行 CTA 重排（直至 L2 缓存容量，参见 Section 3.3）以减少停顿。对于因果掩码，我们额外以降序启动 KV 块，以从对角线开始的升序遍历查询块，并按降序的查询块索引排列 dQ 规约。这种“最短处理时间优先”（SPT）调度确保没有 CTA 在其第一次 dQ 写操作时被阻塞。

3.3 调度

在许多场景中，例如使用因果掩码或可变序列长度 (varlen) 时，注意力内核天然存在负载不均衡——分配给 SM 的工作块的主循环长度不同，因为某些工作块比其他块需要更多的加载和 MMA 操作。此外，我们可以选择 SM 处理块的顺序，例如通过定义网格坐标的首选线性化方式。抽象掉注意力的任何特定功能，我们可以将关于同构并行处理器的最大完工时间 (makespan) 最小化的一般结果应用于我们的场景。特别地，在 FlashAttention-4 中，我们使用经典的最长处理时间优先 (LPT) 调度 [9]。我们强调，我们应用这一思想的方式适用于所有 GPU 架构，并且在 Hopper GPU 上也被验证为 FlashAttention-3 的改进。

因果掩码的 LPT 调度。标准注意力网格为 (m 块，头，批次)，按从左到右递增的顺序计算。但是分数在对角线上方被掩码掉，因此对于固定的头和批次，SM 最终从最短到最长低效地处理工作块。另一方面，朴素的 LPT 顺序也非最优，因为对于不同批次，主循环 KV 加载不会命中 L2 缓存，而且如果 KV 头超过 L2 缓存容量则会发生缓存抖动。相反，我们始终将批次作为最外层维度处理，并对头进行重排。这意味着我们将头分成不超过 L2 缓存容量的组；分块调度器随后按每组头、逆序的 m 块、组别、最后批次的顺序遍历网格。特别地，对于 MQA 或 GQA，我们总是在改变 m 块之前遍历每个 KV 头的所有查询头。经验上验证了这种 LPT 顺序非常有效；例如，对于 BF16 和头

维度 128，在 H200 GPU 上测量到 MHA 获得 4–8% 的 FLOPS 提升，MQA 8 获得 7–14% 的提升。

可变序列长度的 LPT 调度。对于 varlen，我们还需要处理批次间变化引起的负载不均衡。例如，在解码工作负载中，不同批次可能关注不同数量的上下文；在混合批处理或连续批处理中，某些批次可能处于预填充（prefill）阶段而其他处于解码阶段。每个批次的查询和 KV 序列长度列表通常作为注意力元数据存储在设备上，标准的 varlen 注意力内核在以递增顺序处理批次时在运行时读取这些整数。然而，给定的批次顺序在负载均衡方面可能任意地次优——例如，我们可能有较短的方形预填充后跟长上下文解码。为改善这一点，我们可以对批次强制实施 LPT 顺序，通过启动一个预处理内核按最大每工作块执行时间对批次进行排序，写出虚拟批次索引到实际批次索引的额外元数据映射，随后注意力内核按排序顺序读回并遍历批次。这些元数据可以被缓存，因此排序不会带来性能损耗。

4 实现语言与框架

我们完全使用嵌入 Python 的 CuTe-DSL [21] 编写 FlashAttention-4，没有任何 CUDA C++ 组件。CuTe-DSL 编译器接受 Python 源代码，降级到 PTX，然后使用 PTX 编译器（ptxas）最终生成汇编代码（SASS）。

完整表达能力与简洁抽象。CuTe-DSL 编程模型与 CUTLASS C++ 同构，确保，保留了底层 GPU 编程的完整表达能力。CuTe-DSL 提供对 PTX 的直接访问作为逃生舱，允许开发者实现任何所需功能而不受框架限制。例如，我们利用自定义 PTX 序列实现 CuTe-DSL API 中尚未完整暴露的操作（这些将在未来版本中集成），证明了我们的框架不会将开发者限制在有限的 GPU 能力子集中。

通过 JIT 实现快速编译。由于复杂的 C++ 模板元程序，编译时间一直是过去 FlashAttention 实现的瓶颈。通过将 CuTe-DSL 嵌入 Python 并采用即时（JIT）编译，++ 模板的方法实现了更快的构建速度。如表 4 所示，20–30 倍。这种快速迭代周期显著提升了开发者的生产力，在内核开发过程中支持更快的实验和调试。

方法	前向传播	后向传播
FlashAttention-3	55s	45s
FlashAttention-4	2.5s	1.4s
加速比	22×	32×

Table 4: 单个内核的编译时间：FA3（C++ 模板）与 FA4（CuTe-DSL）。通常 FA2 和 FA3 需要为不同注意力变体预编译数百个内核。

灵活性与可访问性。基于 Python 的框架在实践中已展现出其灵活性：开发者已成功在，而无需修改核心框架。通过降低入门门槛，我们的方法使具有几个月 GPU 编程经验的研究人员和工程师在无需 C++ 模板元编程深厚专业知识的情况下，即可贡献有意义的扩展。这种可访问性加速了创新，使注意力机制研究社区能够更快速地探索新的算法变体。

我们的愿景是提供一个综合框架，以最优性能构建各种注意力变体。而是将公共功能分解为独立的可组合原语。块稀疏模式、掩码策略、可变序列长度处理和工作调度等操作均作为正交原语暴露，可以自由组合。这种模块化设计确保优化和新特性惠及所有基于框架构建的注意力实现，同时通过编译为高效 GPU 内核仍然实现最高性能。

5 实证评估

我们评估。

注意力基准测试。我们在不同序列长度和头维度下测量，将其与 PyTorch 标准实现、FlashAttention-2¹、Triton（使用特定于 B200 的指令 [26]）、Gluon（一种比 Triton 具有更精细控制的底层 GPU 编程

¹200

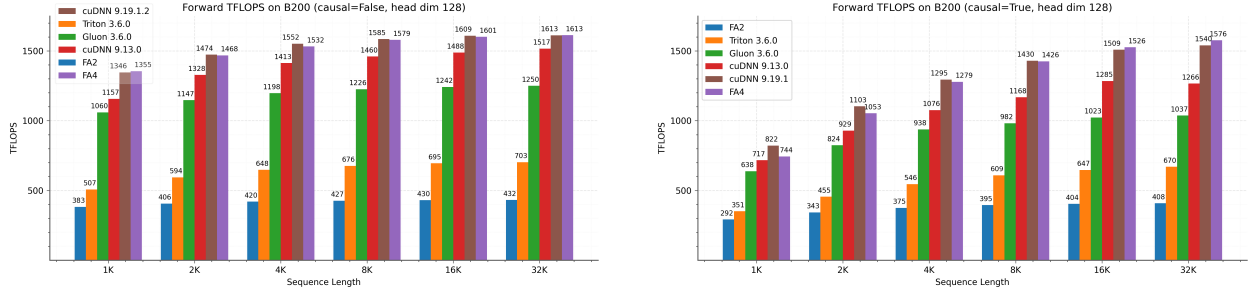


Figure 4: B200 (FP16/BF16) 前向传播 TFLOPS，头维度 128。左：非因果注意力。右：因果注意力。在各序列长度上，FA4 比 cuDNN 9.13.0 实现 1.1–1.3 倍加速，比 Triton 实现 2.1–2.7 倍加速。自我们实现的初始发布以来，cuDNN 的新版本已纳入本文描述的许多技术，性能与 FA4 相近。

语言 [25]) 以及 cuDNN (针对 B200 GPU 优化的厂商库) 进行比较。我们确认 9.13 最高快 1.3 倍，比 Triton 最高快 2.7 倍。200 GPU 上达到最高 1613 TFLOPS/s，约为理论最大 TFLOPS/s 的 71%。

基准测试设置。 我们在 B200 GPU 上针对不同设置 (是否使用因果掩码，头维度 64、128 和 (192, 128)) 对 BF16 输入测量运行时间。序列长度从 1k、2k、... 至 32k，批次大小设置为总 token 数为 32k。隐藏维度设为 2048，头维度为 64 或 128 (即 32 个头或 16 个头)。对于 DeepSeek V3 [7] 使用的 (192, 128) 配置，我们使用 16 个头，查询维度为 192，键值维度为 128。前向传播的 FLOP 计算公式为 $4 \cdot \text{seqlen}^2 \cdot \text{head dimension} \cdot \text{number of heads}$ 。对于因果掩码，我们将此数除以 2，以考虑大约只有一半条目被计算的事实。后向传播的 FLOP 通过将前向传播 FLOP 乘以 2.5 得到 (因为前向传播有 2 次矩阵乘法，后向传播有 5 次，由于重新计算)。

5.1 前向传播

我们在图 ?? 中报告前向传播结果，显示 9.13 快 1.1–1.3 倍，比 Triton 快 2.1–2.7 倍。对于中等和长序列 (4k 及以上)，。因果情况下的提升更大，我们将其归因于最长处理时间优先 (LPT) 调度器。

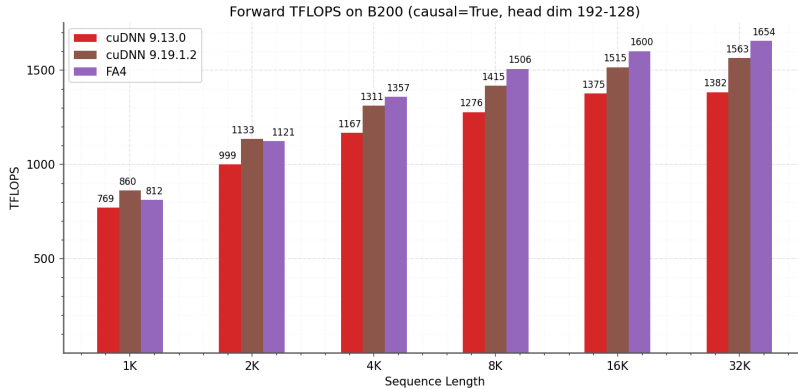


Figure 5: B200 (FP16/BF16) 因果注意力下 cuDNN 与 FA4 前向传播 TFLOPS 比较，头维度 (192, 128) (DeepSeek V3 架构中常用)

5.2 后向传播

我们在图 6 中报告后向传播结果，证明了我们 2-CTA 后向传播的有效性。

我们还在图 7 中展示了确定性后向传播的性能。我们精心设计的数据重排和调度使确定性后向传播速度大幅提升，达到 1-CTA 非确定性后向传播速度的 75%。

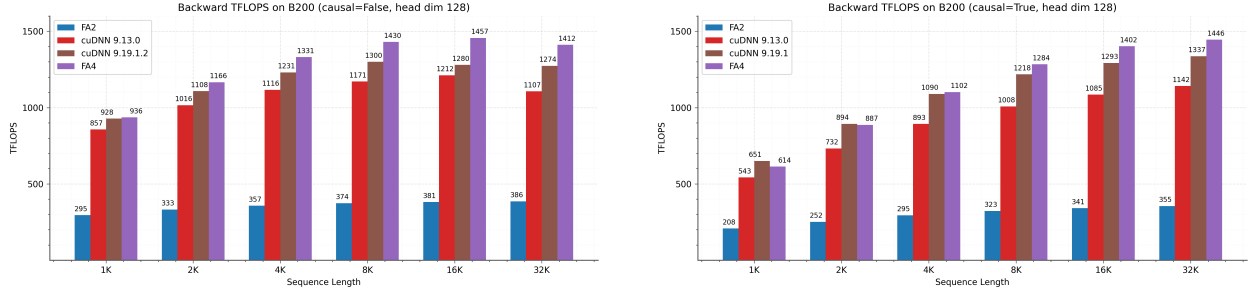


Figure 6: B200 (FP16/BF16) 后向传播 TFLOPS，头维度 128。左：非因果注意力。右：因果注意力。

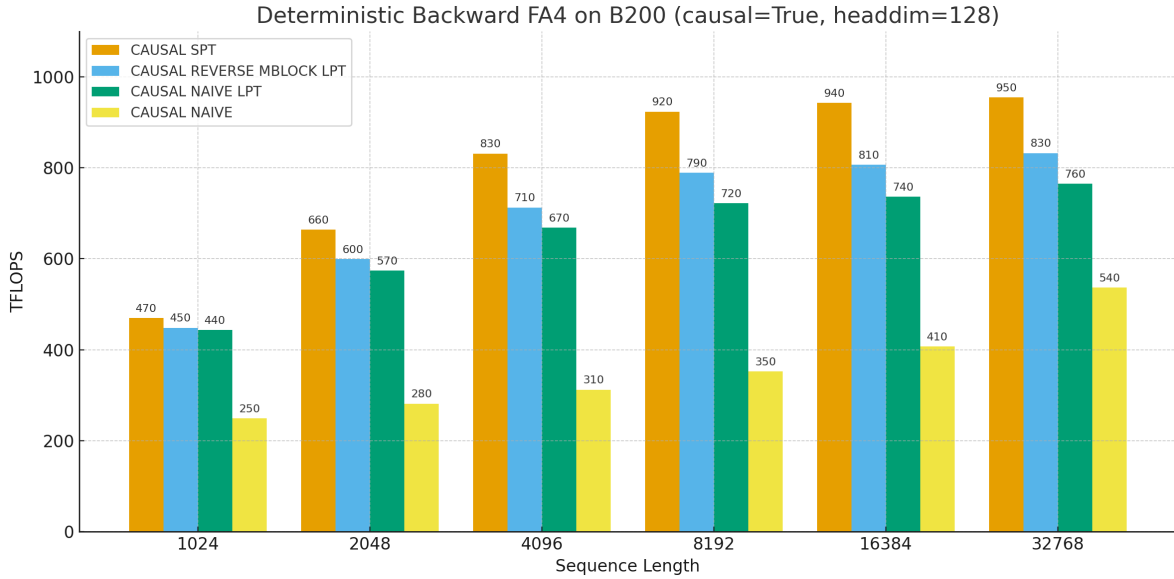


Figure 7: B200 (FP16/BF16) 确定性后向传播消融实验，头维度 128。因果注意力——SPT、逆序 m 块的 LPT、LPT 以及无批次/头重排的朴素方法。

6 讨论与结论

FlashAttention-4 针对非对称硬件扩展问题：由于张量核心的速度已非常快，主要瓶颈转移到了共享内存流量和指数运算吞吐量上，这促使我们进行算法和内核协同设计以缓解这些限制。我们围绕完全异步 MMA 重新设计了流水线，以将 softmax 与更大分块的矩阵乘法重叠，并引入了软件模拟指数运算和条件式 softmax 重缩放以减少非矩阵乘法操作。我们利用张量内存和 2-CTA MMA 模式减少共享内存流量。此外，2-CTA 模式使全局原子累积的重构成为可能，将全局原子加操作的数量减半。FlashAttention-4 完全使用嵌入 Python 的 CuTe-DSL 实现，在保持底层控制的同时实现了比基于 C++ 模板的内核快 20–30 倍的编译速度。尽管针对 Blackwell GPU 进行了优化，随着计算能力持续超越非矩阵乘法单元，这些算法中的一些可以扩展到其他加速器。

致谢

感谢 Together AI、Meta、xAI 和普林斯顿语言与智能中心 (PLI) 提供的算力支持。衷心感谢 Schmidt Sciences AI2050 奖学金、Google ML 与系统青年教职奖以及 Google Research Scholar 项目的支持。还要感谢 NVIDIA 的以下团队：cuDNN、TensorRT-LLM 和 Cutlass 团队不断的讨论、想法和反馈。

References

- [1] Michael Bauer, Henry Cook, and Brucek Khailany. CudaDMA: Optimizing GPU Memory Bandwidth via Warp Specialization. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '11, New York, NY, USA, 2011. Association for Computing Machinery. ISBN 9781450307710. doi: 10.1145/2063384.2063400. URL <https://doi.org/10.1145/2063384.2063400>.
- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [3] Richard J Chen, Chengkuan Chen, Yicong Li, Tiffany Y Chen, Andrew D Trister, Rahul G Krishnan, and Faisal Mahmood. Scaling vision transformers to gigapixel images via hierarchical self-supervised learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16144–16155, 2022.
- [4] Sylvain Chevillard, Mioara Joldeş, and Christoph Lauter. Sollya: An environment for the development of numerical codes. In *Mathematical Software – ICMS 2010*, volume 6327 of *Lecture Notes in Computer Science*, pages 28–31. Springer Berlin Heidelberg, 2010.
- [5] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning, 2023. URL <https://arxiv.org/abs/2307.08691>.
- [6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.
- [7] DeepSeek-AI. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2020.
- [9] R. L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429, 1969. doi: 10.1137/0117039.
- [10] Mandy Guo, Joshua Ainslie, David Uthus, Santiago Ontanon, Jianmo Ni, Yun-Hsuan Sung, and Yinfei Yang. Longt5: Efficient text-to-text transformer for long sequences. *arXiv preprint arXiv:2112.07916*, 2021.
- [11] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J Fleet. Video diffusion models. *Advances in Neural Information Processing Systems*, 35:8633–8646, 2022.
- [12] Jintao Lin, Mingye Luo, Jingwen Zeng, Jianfei Zhai, and Jun Hu. Sageattention2: Efficient attention with thorough outlier smoothing and per-thread int4 quantization. *arXiv preprint arXiv:2411.10958*, 2024.
- [13] Jintao Lin, Jia Zhang, Haofeng Zheng, Jingwen Zeng, Jianfei Zhai, and Jun Hu. Sageattention: Accurate 8-bit attention for plug-and-play inference acceleration. *arXiv preprint arXiv:2410.02367*, 2024.

- [14] Jintao Lin, Mingye Luo, Jia Zhang, Zheng Xu, Haofeng Zheng, Jingwen Zeng, Jianfei Zhai, and Jun Hu. Sageattention3: Fast attention on blackwell gpus via fp4 quantization. *arXiv preprint arXiv:2505.11594*, 2025.
- [15] Weile Luo, Ruibo Fan, Zeyu Li, Dayou Du, Hongyuan Liu, Qiang Wang, and Xiaowen Chu. Dissecting the nvidia hopper architecture through microbenchmarking and multiple level analysis. *arXiv preprint arXiv:2501.12084*, 2025.
- [16] Jean-Michel Muller, Nicolas Brunie, Florent de Dinechin, Claude-Pierre Jeannerod, Mioara Joldes, Vincent Lefèvre, Guillaume Melquiond, Nathalie Revol, and Serge Torres. *Handbook of Floating-Point Arithmetic*. Birkhäuser Boston, 2nd edition, 2018. ISBN 978-3-319-76525-9.
- [17] NVIDIA. Nvidia H100 tensor core GPU architecture, 2022.
- [18] NVIDIA. CUDA Programming Guide Version 12.4, 2024. URL <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>.
- [19] NVIDIA. Nvidia Blackwell architecture technical brief, 2024. <https://nvdam.widen.net/s/xqt56df1gh/nvidia-blackwell-architecture-technical-brief>.
- [20] NVIDIA. cudnn release notes. <https://docs.nvidia.com/deeplearning/cudnn/backend/latest/release-notes.html>, 2025.
- [21] Nvidia. Nvidia cutlass documentation, 2025. URL https://docs.nvidia.com/cutlass/media/docs/pythonDSL/cute_dsl_general/dsl_introduction.html.
- [22] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- [23] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision, 2024. URL <https://arxiv.org/abs/2407.08608>.
- [24] Uri Shaham, Elad Segal, Maor Ivgi, Avia Efrat, Ori Yoran, Adi Haviv, Ankit Gupta, Wenhan Xiong, Mor Geva, Jonathan Berant, et al. Scrolls: Standardized comparison over long language sequences. *arXiv preprint arXiv:2201.03533*, 2022.
- [25] Triton Team. Gluon: A lower-level gpu programming language. <https://github.com/triton-lang/triton/blob/main/python/tutorials/gluon/01-intro.py>, 2024.
- [26] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.
- [27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

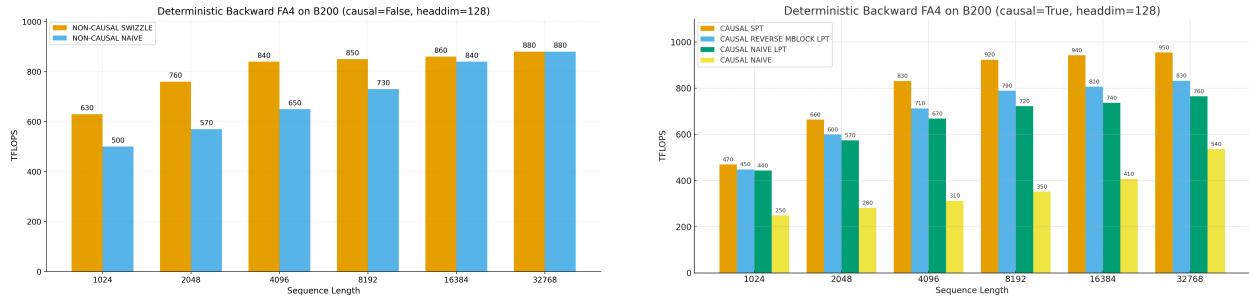


Figure 8: B200 头维度 128 下确定性后向传播消融实验。左：无因果注意力（带/不带批次/头重排）。右：因果注意力——SPT、逆序 m 块的 LPT、LPT 以及无批次/头重排的朴素方法。

A 实验与基准测试补充细节

A.1 系统与库

我们在 B100 180GB SXM6 (1000W) 上对速度进行基准测试。预热 5 次后，重复基准测试 10 次，取平均计时。

我们通常使用撰写本文时（2025 年 3 月）各库的最新版本。具体使用版本：

- CUDA 13.1
- FlashAttention 2.8.3
- Triton 3.6
- PyTorch 2.10.0
- CuTe-DSL 4.4.1

对于 cuDNN，主论文中我们与 cuDNN 9.13 和最新版本 cuDNN 9.19.1.2 进行比较。从 9.13 和 9.14 版本起 [20]，我们与 cuDNN 团队合作，将，使我们的工作能惠及尽可能多的实践者。

A.2 确定性非因果后向传播

为完整起见，我们在图 8 中展示了无因果掩码条件下确定性后向内核的性能，与因果掩码情况并排对比。